

Orbit

Continuous Navigation via Actor-Critic Methods using Neural Function Approximation

Stochastic Batman

March 16-17, 2026

Introduction

This document outlines the thinking process and systematic design for **Orbit**. It serves as a bridge between the theoretical foundations established in *Mathematical Foundations of Reinforcement Learning* by Shiyu Zhao and the practical implementation of neural function approximation in a continuous coordinate space.

1 Environment Specification (`env.rs`)

The environment is modeled as a Markov Decision Process (MDP) where the agent must learn to navigate toward a target in a bounded 2D plane.

1.1 State Space $\mathcal{S}$

The state space is defined over a four-dimensional hypercube. To facilitate stable training in neural networks, all coordinates are normalized to the range $[-1, 1]$. I define the state vector $s \in \mathcal{S}$ as:

$$s \doteq [x, y, x_{target}, y_{target}]^\top \in [-1, 1]^4$$

To maintain consistency in subsequent notation, I denote the components of any state s as:

- $s[0] = x$ (Agent's horizontal position) and $s[1] = y$ (Agent's vertical position)
- $s[2] = x_{target}$ (Target's horizontal position) and $s[3] = y_{target}$ (Target's vertical position)

Notation Note: I use the shorthand $s[i, \dots, k]$ to represent a sub-vector consisting of elements from index i to k inclusive. For example, $s[0, 1]$ refers to the agent's coordinate pair $[s[0], s[1]]^\top$.

1.2 Action Space $\mathcal{A}$ and Transition Dynamics

I employ a discrete action space $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$. Each action corresponds to a displacement vector $\Delta a \in \mathbb{R}^2$ with a step size magnitude $\Delta = 0.05$. The physical directions follow the clockwise convention:

$$\mathcal{A} = \left\{ \overset{\text{North}}{a_1}, \overset{\text{East}}{a_2}, \overset{\text{South}}{a_3}, \overset{\text{West}}{a_4}, \overset{\text{Stay}}{a_5} \right\}$$

The state transition probability $p(s'|s, a)$ is deterministic. The next state s' is calculated by applying the displacement to the agent's coordinates $s[0, 1]$ and clipping the result to the boundary:

$$s'[i] = \text{clip}(s[i] + \Delta a_i, -1, 1), i \in \{0, 1\}$$

The target coordinates remain stationary: $s'[2, 3] = s[2, 3]$.

1.3 Reward Function $r(s, a)$

The reward signal provides dense feedback based on the Euclidean distance between the agent and the target. I define the distance function between any two points p_1, p_2 as:

$$\text{dist}(p_1, p_2) \doteq \sqrt{(p_1[0] - p_2[0])^2 + (p_1[1] - p_2[1])^2}$$

The reward function is formulated to ensure the boundary penalty is strictly lower than any possible reward achievable within the valid state space. In the square $[-1, 1]^2$, the maximum possible distance (the diagonal) is $\sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.8284$.

I define the penalty as -2.85 . This extra ≈ -0.02 is to ensure that a boundary collision is always perceived by the Critic as a "worse" state than being at the furthest possible valid coordinate from the target.

$$r(s, a) = \begin{cases} -2.85 & \text{if } s[0, 1] + \Delta a \notin [-1, 1]^2 \text{ (Boundary Crossing)} \\ -\text{dist}(s'[0, 1], s'[2, 3]) & \text{otherwise} \end{cases}$$

1.4 Termination and Episodes

The task is episodic with the following conditions:

- **Success Condition:** The episode terminates when $\text{dist}(s[0, 1], s[2, 3]) < 0.05$.
- **Truncation Condition:** An episode is capped at $T_{max} = 250$ steps.
- **Terminal Value:** Following the book's convention, the value of the terminal state $v(s_{terminal})$ is defined as 0.

2 Reinforcement Learning Framework

Following the Actor-Critic framework (Zhao, Chapter 10), I decouple the neural architectures from the training logic to ensure modularity.

2.1 Neural Architectures (`model.rs`)

I define the "blueprints" for the function approximators using the `dfdx` type-system. Both networks receive the state $s \in [-1, 1]^4$.

- **Actor** π_θ : A 3-layer MLP mapping states to 5 action logits.
- **Critic** v_w : A 3-layer MLP mapping states to a scalar value estimate.

2.2 The Actor-Critic Agent (agent.rs)

The `OrbitAgent` manages the interaction between the networks and the environment’s feedback loop.

2.2.1 Temporal Difference and Advantage

We utilize the TD error δ_t as an estimate of the advantage $A(s, a)$:

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w)$$

The agent handles termination by setting $\hat{v}(S_{t+1}, w) = 0$ if `is_done` is true. For `is_truncated`, we bootstrap to preserve the infinite-horizon MDP assumption.

2.2.2 Objective Functions

The agent performs two distinct gradient updates per step:

1. **Value Update:** Minimizes $L(w) = \frac{1}{2}\delta_t^2$ via backpropagation through the Critic.
2. **Policy Update:** Minimizes the weighted negative log-likelihood with entropy regularization:

$$L(\theta) = -\ln \pi(A_t|S_t, \theta) \delta_{t,\text{stop_grad}} - \beta H(\pi)$$

where $\delta_{t,\text{stop_grad}}$ indicates that the TD error is treated as a constant scalar during the actor’s backpropagation. ”-” before $\ln$ is because `dfdx` minimizes instead of maximizing.

2.3 Optimization and Hyperparameters

The parameters θ and w are updated using the Adam optimizer. To ensure stable convergence, I employ a two-time-scale update rule where the Critic typically learns faster than the Actor to provide a stable baseline for the advantage signal:

- **Discount Factor (γ):** 0.99 (Prioritizing long-term navigation).
- **Learning Rates:** $\alpha_\theta = 1 \times 10^{-4}$ (Actor), $\alpha_w = 1 \times 10^{-3}$ (Critic).
- **Entropy Regularization:** To prevent premature convergence to a sub-optimal deterministic policy, I may add an entropy term $H(\pi)$ to the Actor loss: $L_{total}(\theta) = L(\theta) - \beta H(\pi(s, \theta))$ where $\beta = 0.01$.

2.4 Softmax Policy Derivation

For the discrete action set $\{a_1, \dots, a_5\}$, the probability of selecting action a_i given logits z from the Actor is:

$$\pi(a_i|s, \theta) = \frac{e^{z_i(s, \theta)}}{\sum_{j=1}^5 e^{z_j(s, \theta)}}$$